
INTERNSHIP PROJECT REPORT

ON

Telecom Support Workflow Executor with LLM-Assisted Decision Routing

Submitted in partial fulfillment of the requirements for the internship at

Jio Platforms Limited



Github Repository : github.com/adilnvm/Workflow-Executor

Submitted By: Adil Iqbal (Intern)

Team : Jio EVA (AuthTool)

Mentor: Mr. Ankit Shinde

Internship Duration: 21/08/2025 - 31/12/2025

TABLE OF CONTENTS

1. Abstract	3
2. Problem Statement	4
3. Project Objectives	5
4. System Architecture	6
○ 4.1 Architectural Overview	6
○ 4.2 Core Components	6
○ 4.3 Architecture Diagram	7
5. Workflow Design and Implementation	7
○ 5.1 Network Troubleshooting Workflow	7
○ 5.2 Billing Explanation Workflow	8
○ 5.3 Recharge Resolution Workflow	8
○ 5.4 SIM / Device Troubleshooting Workflow	8
6. Execution Model and Ticket Lifecycle Management	9
○ 6.1 The Ticket Lifecycle	9
○ 6.2 Slot-Based Execution Gating	10
○ 6.3 Clarification Handling Mechanism	10
7. Failure Handling and System Robustness	11
○ 7.1 LLM Reliability Strategies	11
○ 7.2 Storage Resilience	11
○ 7.3 Rollback Mechanism	12
8. Metrics and Performance Evaluation	12
○ 8.1 Tracked Metrics	12
○ 8.2 Example Data Analysis	13
9. Frontend Implementation	13
10. System Limitations	14
11. Future Work and Enhancements	14
12. Conclusion	15

1. Abstract

This project focuses on designing and implementing a deterministic workflow execution system for telecom customer support. Unlike traditional conversational chatbots that rely heavily on probabilistic generation for both understanding and answering, this system utilizes a Large Language Model (LLM) strictly for **intent classification** and **decision routing**. All actual troubleshooting logic is executed through deterministic, auditable workflows.

The core motivation was to reduce user friction, eliminate unnecessary clarifications, and avoid the unpredictable behavior (“hallucinations”) commonly observed in generative AI support agents. The system enforces strict architectural invariants where LLM outputs are treated as **untrusted input**, validated through Pydantic schemas, and never allowed to directly perform system actions. State is explicitly owned by the backend through a custom ticketing system that preserves context across turns and survives application restarts.

Slot-based execution gates ensure that workflows only run when execution-critical information is available, asking targeted clarification questions only when necessary. The architecture is designed to degrade gracefully in the presence of LLM failures, infrastructure outages (e.g., Redis downtime), or partial user input. The resulting system behaves like a production-grade Tier-1 telecom support engine rather than a conversational assistant, emphasizing correctness, predictability, and operational safety.

2. Problem Statement

Telecom customer support systems handle extremely high volumes of repetitive, structured issues such as network outages, billing queries, recharge failures, and SIM/device troubleshooting.

The Limitations of Traditional Approaches:

1. **Standard Chatbots (Generative AI):**

- **Ambiguity:** They often treat every user message as a new query, leading to loops where the user must repeat information.
- **Hallucinations:** Generative models may invent policies, refund amounts, or technical explanations that do not exist.
- **Lack of Auditability:** It is difficult to trace *why* a chatbot gave a specific answer if the logic is hidden inside a neural network.

2. Manual Support (Human Agents):

- **Cost & Latency:** Human agents are expensive and cannot scale instantly during outage spikes.
- **Inconsistency:** Different agents may follow troubleshooting steps differently.

The Need: There is a clear gap for a system that combines the **intent understanding** of LLMs with the **reliability** of traditional code. The system must:

- Use AI only where probabilistic reasoning is acceptable (understanding the user).
- Execute troubleshooting deterministically (solving the problem).
- Preserve context (remembering the user's region or account type).
- Provide safe fallback and escalation paths when the AI fails.

3. Project Objectives

The primary objective was to build a **Workflow Executor**, not a chatbot. Specific goals included:

- **Architectural Separation:** Decouple the "Brain" (LLM) from the "Hands" (Tools). The LLM decides *what* to do, but Python code decides *how* to do it.
- **Deterministic Execution:** Ensure that given the same inputs (facts), the system always produces the same troubleshooting result.
- **Minimizing Friction:** Implement a slot-filling mechanism that only asks for missing information required to solve the specific problem, avoiding open-ended "How can I help?" loops.
- **State Persistence:** Implement a robust storage layer (Redis with memory fallback) to handle long-running tickets and server restarts without data loss.

- **Graceful Degradation:** Ensure the system continues to function (or fails safely) if the LLM provider is down, the database is unreachable, or the user provides garbled input.
 - **Observability:** Instrument the system to track metrics like escalation rates and missing slots to drive future improvements.
-

4. System Architecture

4.1 Architectural Overview

The system follows a modular, layered architecture designed for production readiness. It is built using **FastAPI** for the web layer, **Pydantic** for rigorous data validation, and a pluggable **LLM provider** layer.

4.2 Core Components

1. **Frontend (Thin UI)**
 - *Technology:* HTML, CSS, Vanilla JavaScript.
 - *Role:* Captures user input and renders the JSON response from the backend. It contains *zero* business logic, ensuring the backend is the single source of truth.
2. **FastAPI Backend (`main.py`)**
 - *Role:* Acts as the entry point/controller. It receives requests, instantiates the agent, and returns structured `WorkflowResponse` objects.
3. **LLM Layer (`llm/`)**
 - *Role:* Strictly limited to **Intent Classification** and **Entity Extraction**.
 - *Design:* It uses an abstract base class (`BaseLLM`) to allow hot-swapping between `GeminiLLM` (Production), `Phi3LLM` (Local/Dev), and `MockLLM` (Testing).
 - *Safety:* All LLM outputs are forced into a strict JSON schema defined in `schemas/decision.py`.
4. **Ticket State & Persistence (`storage/`)**
 - *Role:* Maintains the "Memory" of the conversation.
 - *Innovation:* Implements a **Hybrid Store**. The system primarily connects to **Redis** for persistence. If Redis is unavailable, it automatically falls back to an

InMemoryTicketStore without crashing. This ensures high availability during development or infrastructure failures.

5. **Slot-Based Execution Gate** (**utils/slot_checker.py**)

- *Role:* The "Guardrail". It checks if the current ticket has all the necessary "facts" (slots) required to run the chosen workflow. If a slot is missing, it blocks execution and generates a specific clarification question.

6. **Workflow Engine** (**workflow/, tools/**)

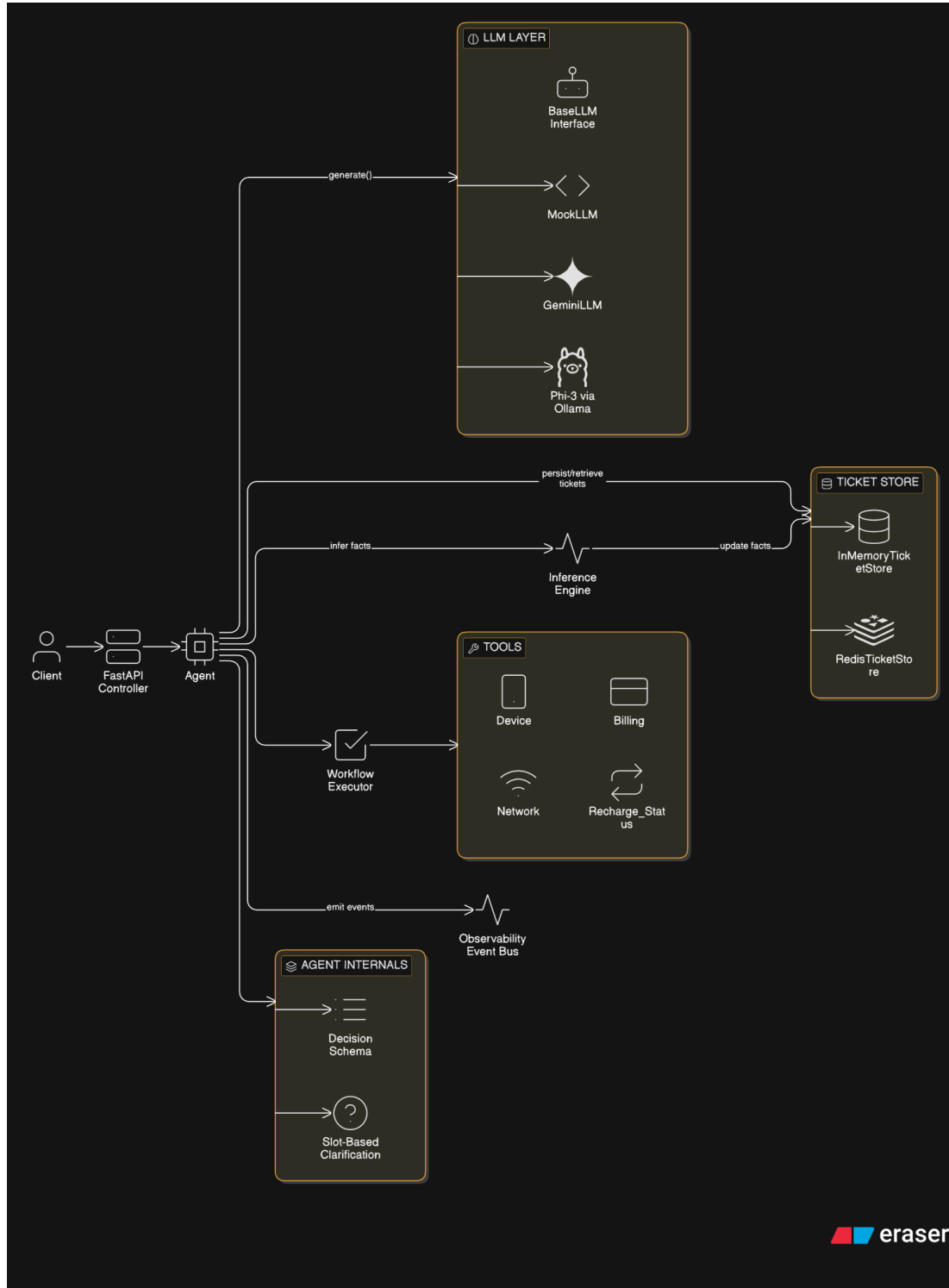
- *Role:* The "Doer". Contains deterministic Python functions that execute business logic (e.g., checking network status endpoints, calculating bill details).

7. **Observability Bus** (**observability/**)

- *Role:* An event-driven bus that decouples logging and metrics from business logic. Events like **ticket_created** or **workflow_escalated** are emitted asynchronously to sinks (Logs, Metrics Collector)

4.3 Architecture Diagram

[Diagram Link](#)



5. Workflow Design

The system implements distinct, deterministic workflows. Each workflow is a Python function that takes a context (dictionary of facts) and returns a resolution.

5.1 Network Troubleshooting Workflow

([workflow/network_workflow.py](#))

- **Trigger:** Intent `slow_internet` or `no_signal`.
- **Required Slot:** `region`.
- **Logic:**
 1. **Outage Check:** Queries `tools.network.check_outage_backend`. If the region (e.g., "East India") has an active outage, return **Resolved** with ETA.
 2. **Congestion Check:** If no outage, queries `check_congestion_backend`. If the region is a high-traffic metro (e.g., Mumbai) during peak hours, return **Resolved** with a congestion notice.
 3. **Escalation:** If neither issue is found, escalate to human support.

5.2 Billing Explanation Workflow ([workflow/billing_workflow.py](#))

- **Trigger:** Intent `billing_issue` or `unexpected_charges`.
- **Required Slot:** `account_type` (Prepaid/Postpaid).
- **Logic:**
 1. Fetches a billing snapshot using `tools.billing.fetch_billing_snapshot`.
 2. If data is missing, return **Escalated**.
 3. If `account_type` is Postpaid, it breaks down the bill (Rent + Usage + Add-ons).
 4. If `account_type` is Prepaid, it explains the last recharge validity and balance.

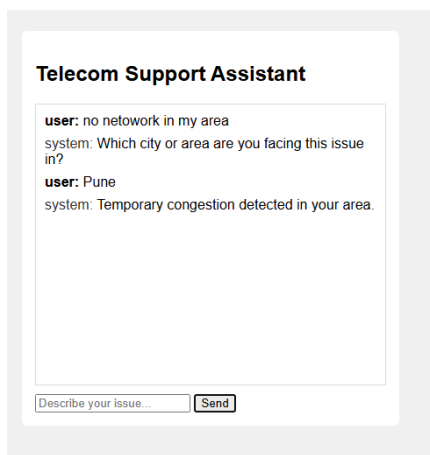
5.3 Recharge Resolution Workflow ([workflow/recharge_workflow.py](#))

- **Trigger:** Intent `recharge_issue`.
- **Required Slot:** `account_type`.
- **Logic:**
 1. Checks `tools.recharge.fetch_recharge_status`.
 2. **Success Delayed:** Explains SLA (e.g., "Wait 30 minutes").
 3. **Failed Charged:** Critical failure mode; returns **Escalated** immediately for refund processing.

5.4 SIM / Device Troubleshooting Workflow

(`workflow/sim_device_workflow.py`)

- **Trigger:** Intent `sim_issue` or `device_issue`.
- **Required Slot:** `device_type`.
- **Logic:**
 1. **Sim Detection:** Checks if the device model (e.g., "iPhone 12") detects the SIM.
 2. **Compatibility:** Checks if the device supports the network frequency (e.g., older Android devices might be unsupported).
 3. Returns specific remediation steps (e.g., "Reinsert SIM", "Upgrade Device") or escalates.
- A very Simple User query and Response:



The screenshot shows a chat window titled "Telecom Support Assistant". Inside the chat area, the following messages are displayed:

- user:** no network in my area
- system:** Which city or area are you facing this issue in?
- user:** Pune
- system:** Temporary congestion detected in your area.

At the bottom of the chat window, there is a text input field labeled "Describe your issue..." and a "Send" button.

6. Execution Model & Ticket Lifecycle

The execution model is the heart of the system, residing in `agent.py`. It ensures that the system behaves like a State Machine rather than a text generator.

6.1 The Ticket Lifecycle

1. **Initialization:** When a user sends a message, a unique `ticket_id` is generated (UUID).
2. **Snapshotting:** Before processing, the current state of the ticket is deep-copied. This allows for **Rollback** if the LLM crashes or produces invalid JSON.
3. **LLM Decision:** The `GeminiLLM` (or fallback) analyzes the message and outputs a `Decision` object containing:
 - `intent`: e.g., "network_issue"
 - `entities`: e.g., {"region": "mumbai"}
 - `workflow`: e.g., "network_troubleshooting_workflow"
4. **Inference (Phase-5):** The `utils.inference_engine` attempts to derive soft facts (e.g., inferring "mobile" service if the user mentions "SIM"). These are marked as reversible and never override user input.
5. **Fact Merging:** New entities are merged into the ticket's `facts` dictionary. **Invariant 4** ensures that stable facts (like `account_type`) are not overwritten by lower-confidence guesses.

The screenshot displays the 'workflow-executor' dashboard. At the top, there's a header with the product name, a star icon, and a menu. Below the header, a navigation bar includes links for 'Free Tier', 'AWS', 'Mumbai, India', 'ap-south-1', and 'Global'. To the right of the navigation bar are links for 'Docs' and 'SDK'. Below the navigation bar, there's a tabbed interface with 'Details' selected. The 'Details' tab shows four resource usage cards: 'COMMANDS' (188 / 500k per month), 'BANDWIDTH' (0 B / 50 GB), 'STORAGE' (11 KB / 256 MB), and 'COST' (\$0.00). Each card has a progress bar and an 'Upgrade' button. Below the resource cards, there's a section for 'Endpoint', 'Token / Readonly Token', 'Port', and 'TLS/SSL'. The endpoint is 'peaceful-treefrog-52577.upstash.io', the token is masked with dots, the port is '6379', and TLS/SSL is 'Enabled'. At the bottom, there's a code block showing the Redis CLI command: 'redis-cli --tls -u redis://default:*****@peaceful-treefrog-52577.upstash.io:6379'.

Resource	Usage	Limit	Status
COMMANDS	188 / 500k per month	500k	Writes: 60, Reads: 128
BANDWIDTH	0 B	50 GB	It's all right.
STORAGE	11 KB	256 MB	It's all right.
COST	\$0.00		

Endpoint	Token / Readonly Token	Port	TLS/SSL
peaceful-treefrog-52577.upstash.io	*****	6379	Enabled

```
redis-cli --tls -u redis://default:*****@peaceful-treefrog-52577.upstash.io:6379
```

6.2 Slot-Based Execution Gating

A major innovation in this project is the rejection of LLM-driven conversation flow in favor of **Slot Gating**.

- **Concept:** A workflow *cannot* start until its **REQUIRED_SLOTS** are full.
- **Mechanism:** `utils.slot_checker.get_missing_slot` scans the ticket facts.
- **Example:**
 - User: "My internet is slow."
 - LLM Intent: `slow_internet`.
 - Workflow: `network_workflow` requires `[region]`.
 - Fact Check: `region` is missing.
 - **Action:** The system halts execution and returns the pre-defined question for `region`: "Which city or area are you facing this issue in?"

```
#slot philosophy
# If workflow can execute → execute
# Else → ask ONE targeted question
```

6.3 Clarification Handling

Unlike chatbots that might ask "Can you tell me more?", this system asks **Deterministic Questions**.

- When a user replies to a clarification (e.g., "Mumbai"), the `utils.clarification_parser` extracts the value directly without invoking the LLM if possible, or uses the LLM to extract just that entity.
- This ensures that every user turn brings the system closer to resolution (Convergence).

7. Failure Handling & Robustness

The system is built on the philosophy of "**Graceful Degradation**". It assumes that every component (LLM, Database, Network) can and will fail.

7.1 LLM Reliability Strategies

- **Schema Enforcement:** The LLM output is parsed by `Pydantic`. If the LLM returns invalid JSON or hallucinates a new intent, the system catches the validation error and triggers a safe fallback (asking the user to rephrase).
- **Provider Fallback:** The `llm_provider.py` can switch from Gemini to a local Phi-3 model if the API quota is exceeded (`QuotaExceededError`) or network connectivity is lost.

7.2 Storage Resilience

- The `StoreProvider` pattern abstracts the database.
- **Primary:** `RedisTicketStore` handles production traffic with TTL (Time To Live) to auto-expire old sessions.
- **Fallback:** If Redis connectivity fails (e.g., connection refused), the system logs a warning and instantly swaps to `InMemoryTicketStore`. The user experiences no downtime, only a loss of persistence if the server process restarts.

7.3 Rollback Mechanism

- If an unhandled exception occurs during workflow execution (e.g., a division by zero in a billing calculation), the system restores the ticket state from the snapshot taken at the start of the request.
 - This **prevents "corrupted tickets"** where a user is stuck in a broken state.
-

8. Metrics & Evaluation

To move beyond anecdotal testing, the system includes a `MetricsCollector` (`metrics/collector.py`).

8.1 Tracked Metrics

- **total_tickets**: Volume of traffic.
- **resolved**: Success rate of deterministic workflows.
- **escalated**: Failure rate (or cases requiring human intervention).
- **slot_missing_frequency**: Which information users forget most often (e.g., users rarely state their **account_type** initially).

8.2 Example Data

The `/metrics` endpoint provides a snapshot of system health:

```
JSON
{
  "total_tickets": 150,
  "resolved": 110,
  "escalated": 40,
  "slot_missing_frequency": {
    "region": 45,
    "account_type": 30
  }
}
```

This data is crucial for optimizing the UX (e.g., if **region** is always missing, we might prompt for it earlier).

9. Frontend Implementation

While the focus was backend engineering, a lightweight functional Frontend was built to demonstrate the API capabilities.

- **Visual Feedback**: Displays user messages (right) and system responses (left).
- **State Tracking**: The frontend holds the **ticket_id** in a JavaScript variable (`app.js`) and sends it with every request, proving the stateful nature of the backend.

- **Escalation UI:** Visual indicators change color when a ticket status returns `escalated`, simulating a handover to a live agent console.
-

10. Limitations

- **Mocked Backends:** The tools (`tools/*.py`) simulate BSS/OSS responses. In a real deployment, these would need to wrap complex SOAP/REST APIs of the telecom provider.
 - **Synchronous Execution:** The current architecture blocks until the workflow finishes. Long-running diagnostics (e.g., a 2-minute line test) would require moving to an asynchronous task queue (e.g., Celery).
 - **Language Support:** The Slot Checker and Entity Extractor are currently optimized for English. Supporting Hindi or Hinglish would require fine-tuning the LLM prompts and updating the `region_mapper`.
-

11. Future Work

1. **Asynchronous Workflows:** Implement a "callback" mechanism where the system can say "Running diagnostics..." and push a notification to the frontend when done.
 2. **RAG Integration:** For "Knowledge" queries (e.g., "How do I activate 5G?"), a Retrieval-Augmented Generation pipeline could be added alongside the deterministic workflows.
 3. **Human-in-the-Loop:** Build an "Agent Dashboard" where escalated tickets appear with their full context (History + Facts) so a human agent doesn't need to ask the same questions again.
 4. **Advanced Observability:** Persist the observability events to a database (e.g., Postgres/Elasticsearch) for long-term trend analysis.
-

12. Conclusion

This project successfully demonstrates how Large Language Models can be integrated into production-critical systems **safely**. By adhering to the "Re-Anchoring" philosophy—treating the LLM as a probabilistic classifier rather than an omniscient brain—we built a system that is robust, predictable, and audit-friendly.

The separation of **Intent (AI)** from **Execution (Code)** allows the system to scale efficiently. The inclusion of architectural safeguards like the **Slot Gate**, **Redis Fallback**, and **Rollback Logic** ensures that the system behaves like a mature enterprise product. This internship provided deep insights into backend systems design, the practical limitations of Generative AI, and the importance of invariants in software engineering.